Principal Enterprise Engineer Case Study

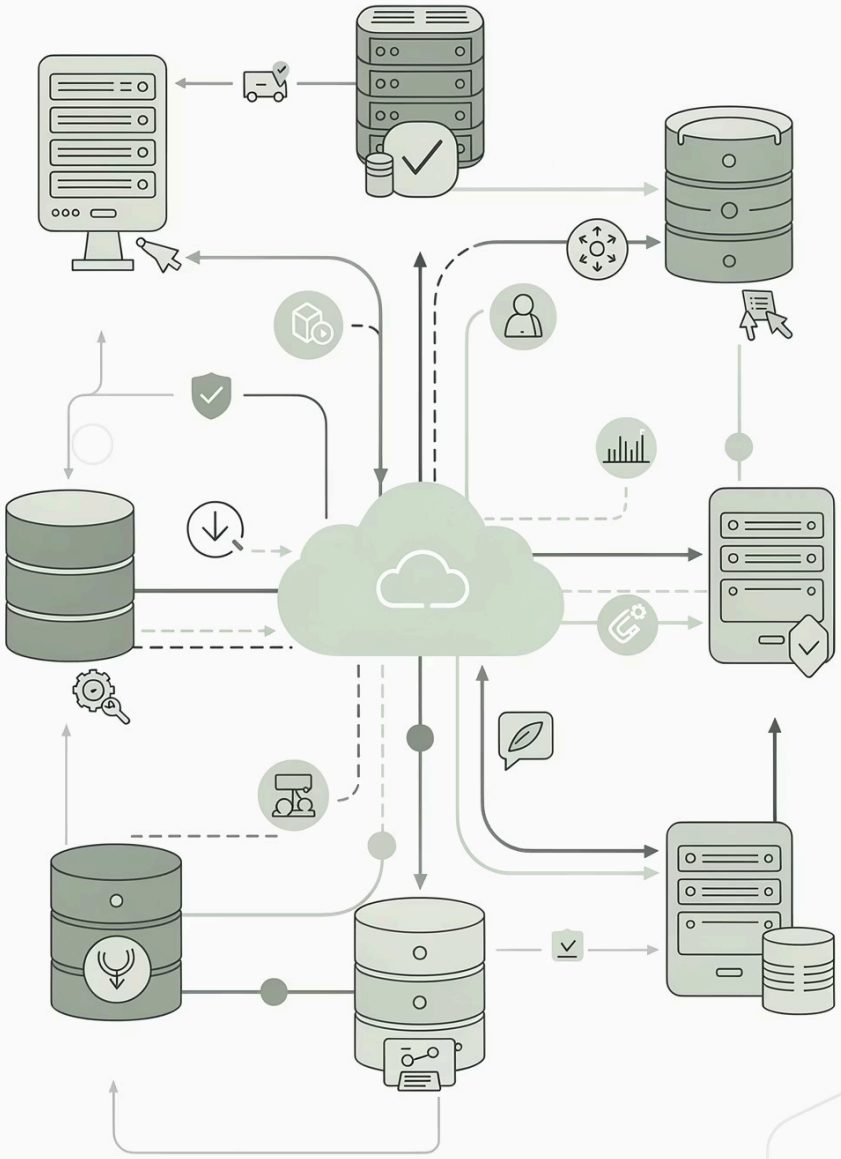
Integration Platform Architecture & Estimation

Presented by: Damian Gonzalez

For: Jim Smitley, VP of Engineering

Company: Albert Invent

Date: February 17, 2026



Estimate and Scoping

15 min

How I'd Approach This Engagement

My first move is NOT to estimate. It's to **separate what we know from what we don't know**.

- **What we know:** The system landscape, directional requirements, the July 1 deadline.
- **What we don't know:** Data volumes, API capabilities, field-level mappings, data quality.

One-Time Migrations (Hard Deadline)

- Biovia ELN (Electronic Lab Notebook) subscription ending
- CISPro (Chemical Inventory System) subscription ending

Ongoing Integrations (Long-Term Reliability)

- LIMS (Laboratory Information Management System) Integration
- Azure Data Platform Integration

My approach in one sentence: Establish a rough range to set budget expectations → run a focused discovery to narrow the range by 60% → commit to a fixed price for the well-defined work and T&M (Time and Materials) for the high-uncertainty work.

Four Workstreams, Two Categories

#	Workstream	Type	Direction	**Deadline**
1	ELN Migration (Biovia ELN → Albert Notebook)	One-time file migration	Inbound	**July 1**
2	LIMS Integration (LabVantage ↔️ Albert LIMS)	Ongoing bidirectional	Bidirectional	Desired
3	Data Warehouse (Albert → Azure)	Ongoing outbound	Outbound	Flexible
4	Inventory Migration (CISPro → Albert Inventory)	One-time structured data	Inbound	**July 1**

Workstreams 1 and 4 are non-negotiable on timing (Biovia shuts off **July 1**). Workstreams 2 and 3 have more flexibility.

T-Shirt Sizing Framework: 6 Key Questions

T-shirt sizing is a standard agile estimation technique documented in Mike Cohn's 'Agile Estimating and Planning' (2005) and widely used in enterprise software for pre-discovery scoping. The 6-question framework below adapts Cohn's relative sizing approach with dimensions specific to integration and migration work.

	Small (S) 1-3 pw	Medium (M) 3-8 pw	Large (L) 8-16 pw	XL 16-32 pw
1. How many systems?	1 system	1-2 systems	2-3 systems	3+ or very complex
2. Directionality?	One-way, read-only	One-way, write	Bidirectional	Bidirectional + state sync
3. Data complexity?	Files or simple flat data	Structured data, moderate mapping	Complex relationships, heavy transformation	Unknown schema + complex rules
4. API availability?	Well-documented REST	Documented but quirky	Poorly documented	No API (must reverse-engineer)
5. Ongoing vs one-time?	One-time, no monitoring	One-time with validation	Ongoing, needs monitoring	Ongoing + SLA/compliance
6. Have we done this before?	Standard pattern	Similar to past work	First time, but familiar tech	First-of-kind

Pre-Discovery Estimate (Build Phase Only)

Initial ranges to set expectations before detailed discovery.

Workstream	Optimistic	Expected	Pessimistic	Key Assumption
1. ELN Migration	4 pw	7 pw	12 pw	Biovia provides clean export; <50K files
2. LIMS Integration	16 pw	22 pw	32 pw	LabVantage API accessible; standard lifecycle
3. Data Warehouse	3 pw	5 pw	9 pw	Customer has established ADF patterns
4. Inventory Migration	6 pw	10 pw	16 pw	CISPro data is reasonably clean; <100K records
Build Subtotal	**29 pw**	**44 pw**	**69 pw**	

(pw = person-weeks)

Full Program Scope

Includes: Discovery (3-4 pw), testing, deployment, and hypercare (2-4 pw/workstream).

Full-Loaded Range

~40-80 person-weeks
(~$250K-$500K at typical enterprise integration rates)

Discovery: From Range to Commitment

Duration & Team

3-4 weeks | Principal Engineer (lead) + 1-2 integration engineers + customer technical leads

Discovery Deliverables

- Technical Discovery Report (per workstream): API assessment, data quality, volume
- Data Mapping Specifications: field-level source → target with transformation rules
- Architecture Diagrams: sequence flows, component diagrams, error handling
- Risk Register + Assumption Log: project-specific, not generic templates
- Detailed PERT Estimate: work-package level with contingency buffers
- Draft SOW (Statement of Work): phases, milestones, commercial terms

Technical Spikes (run during discovery)

- Biovia ELN export test
- LabVantage API connectivity
- CISPro data extract
- Albert SDK validation

Fallback Option: If the customer won't pay for formal discovery, run a 1-week accelerated assessment at Albert's cost. Output: lighter-weight scope document with wider ranges. Price the implementation higher to account for remaining unknowns.

Making This Work: A Shared Risk Model

How We Partner

- Joint discovery workshops
- Process mapping
- Strawman data mappings
- Weekly syncs

Customer Commitments

1. Named PM with authority
2. API access within 2 weeks
3. SME availability 0.5 FTE
4. Test data within 3 weeks
5. 3-day feedback SLA

Timeline

Discovery → Design → Build → Test → Deploy | July 1 deadline for migrations

Triage Plan

- Must: ELN + Inventory
- Can slip: Azure integration
- Should slip: LIMS integration

Three Risks I'd Raise Before We Start

Identifying potential challenges early to ensure a smooth project delivery.

Customer Data Needs Unclear

Impact: Discovery delays, significant scope creep.

Mitigation: Focused discovery with process mapping and strawman data proposals.

Biovia No Public APIs

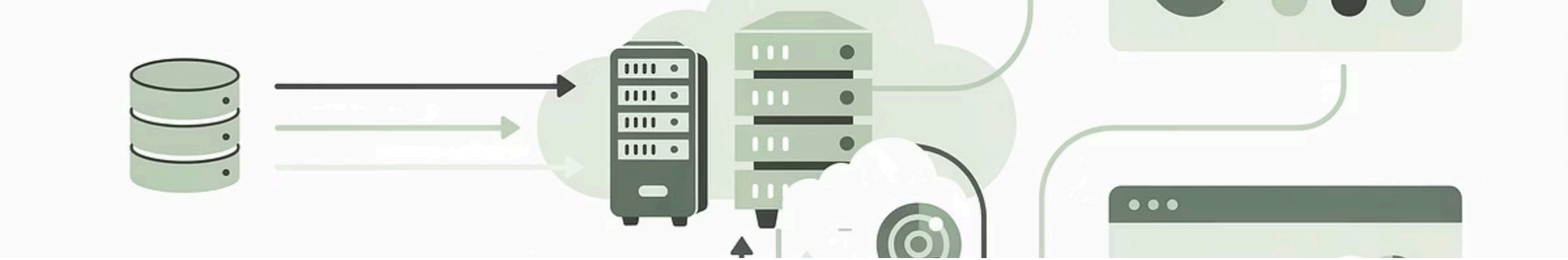
Impact: Migration depends on less common methods (Pipeline Pilot, Oracle DB).

Mitigation: Technical spike in Week 1 to validate extraction path.

First-of-Kind LIMS Integration

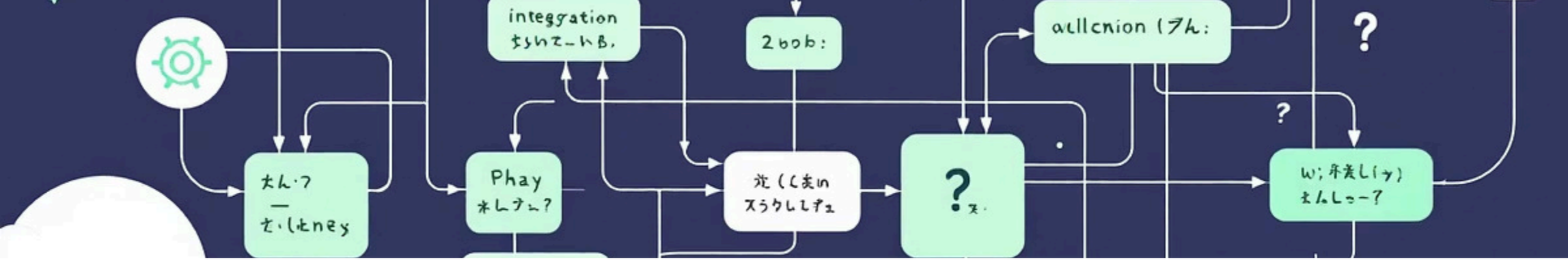
Impact: 25-35% contingency needed for unknowns; no reference architecture.

Mitigation: Meticulous documentation, phased approach, productize post-go-live.



LIMS Integration Plan

20 min



LIMS Integration — What I'm Assuming

1

LabVantage REST API is enabled and accessible from Albert's cloud infrastructure (not blocked by firewall or VPN)

2

The customer has a **LabVantage admin** who can configure server-side queries and RESTPolicy security rules (LabVantage uses deny-by-default — nothing is accessible until explicitly allowed)

3

The integration scope is **request/result exchange** — Albert submits test requests, LabVantage returns measured results. We are NOT migrating historical data or replacing LabVantage's UI.

4

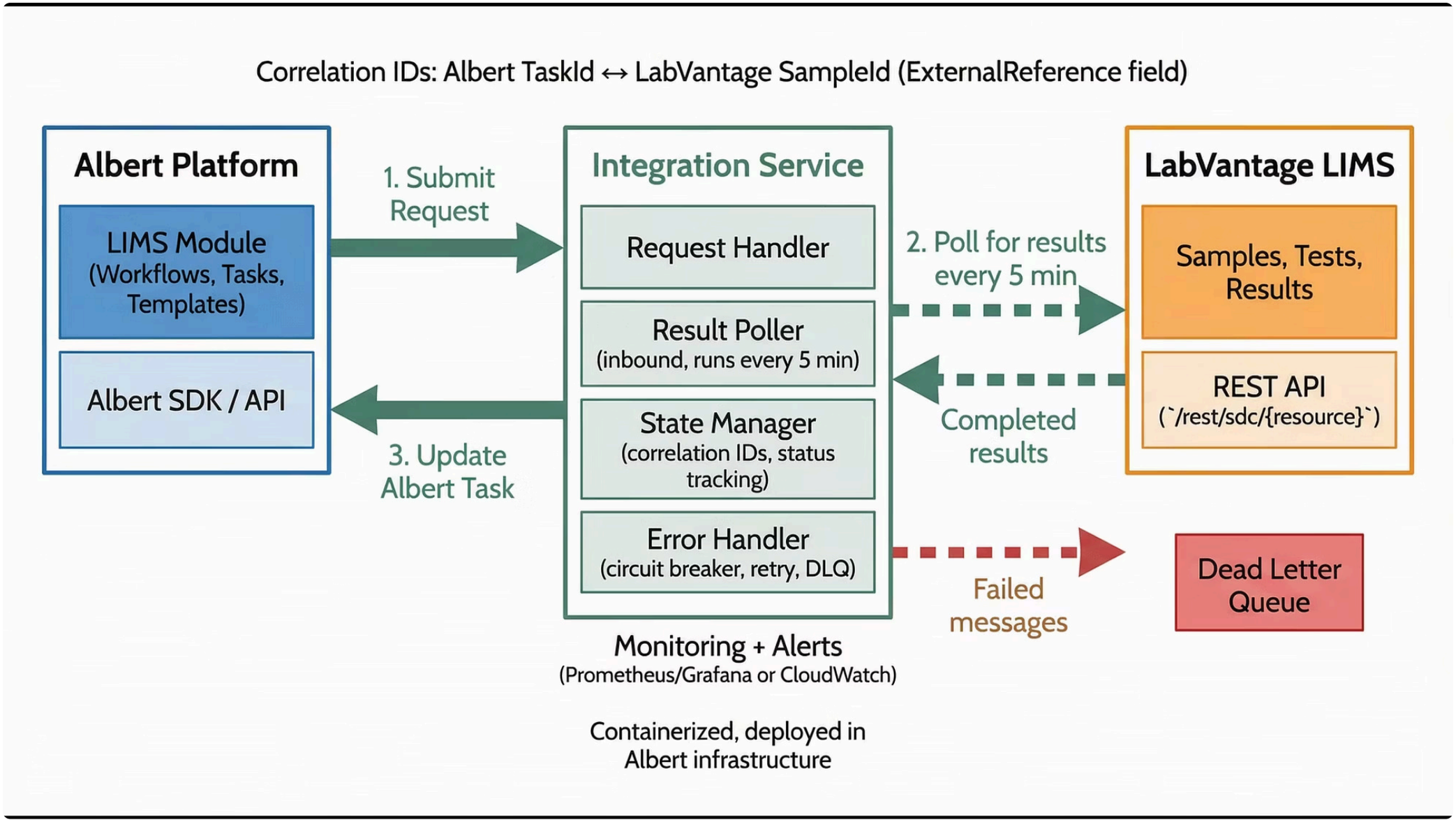
Lab test turnaround times are **hours to days** — this justifies polling (every 5 min) rather than real-time event streaming

5

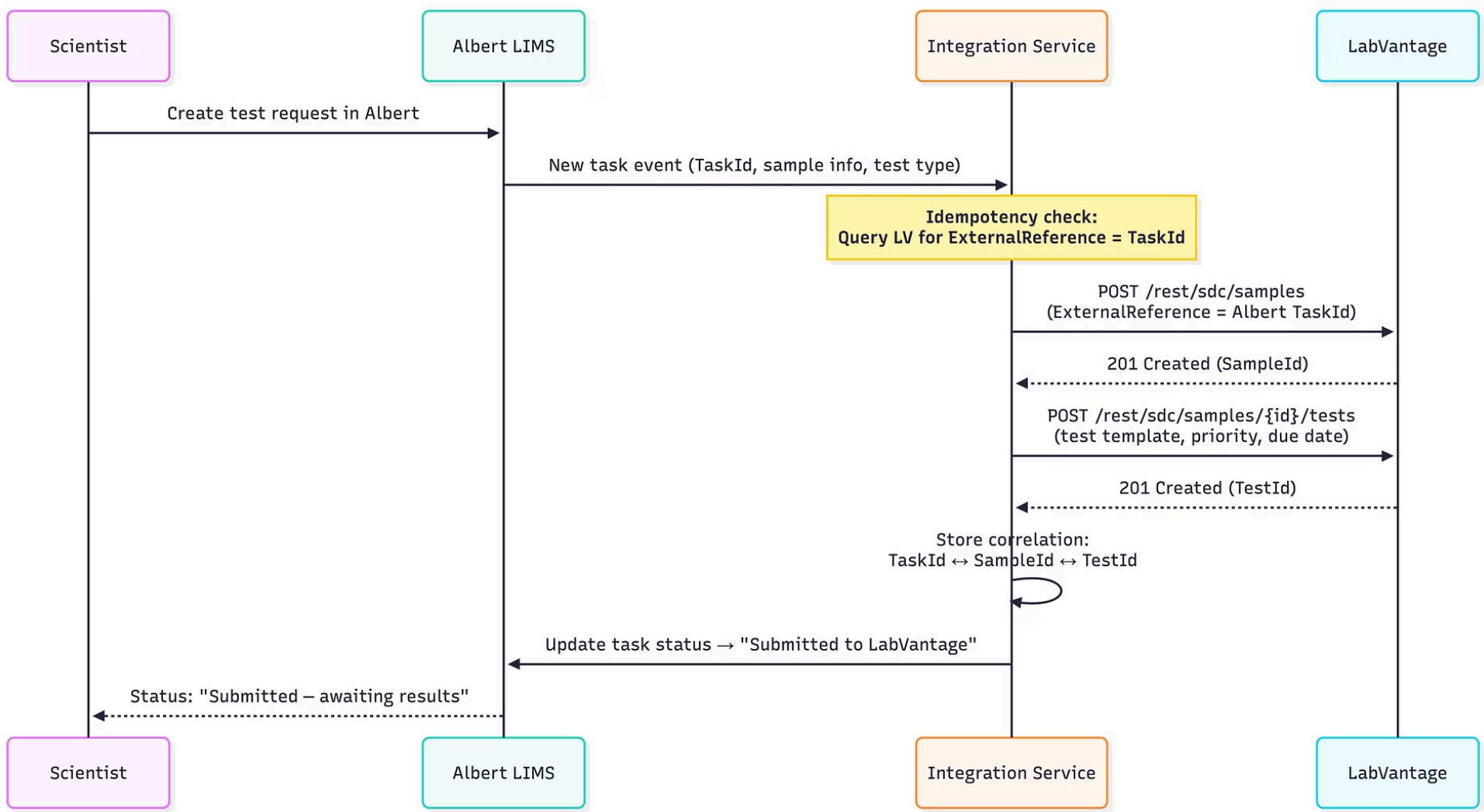
Albert's LIMS module (Workflows, Tasks, Templates) has an **API/SDK** that supports programmatic task creation and result attachment

Each assumption, if false, changes the architecture. These must be validated during discovery.

LIMS Integration Architecture

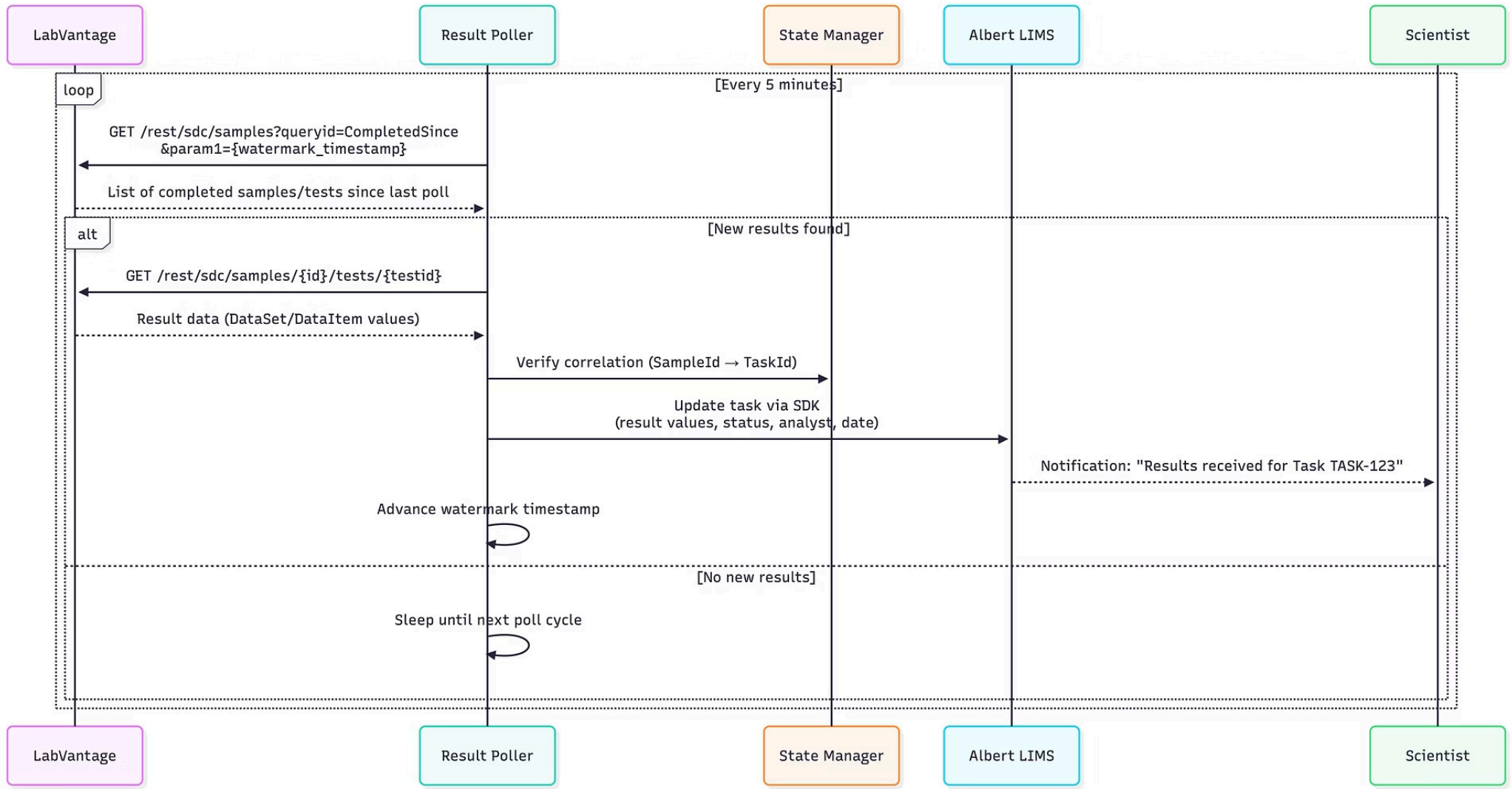


Outbound: Submitting a Test Request



Idempotency Check: Before creating a sample in LabVantage, the Integration Service queries LabVantage to see if a sample with the Albert TaskId (ExternalReference) already exists. If it does, it skips sample creation to prevent duplicate submissions during retries, ensuring data consistency.

Inbound: Receiving Test Results



Why Polling at 5 Minutes?

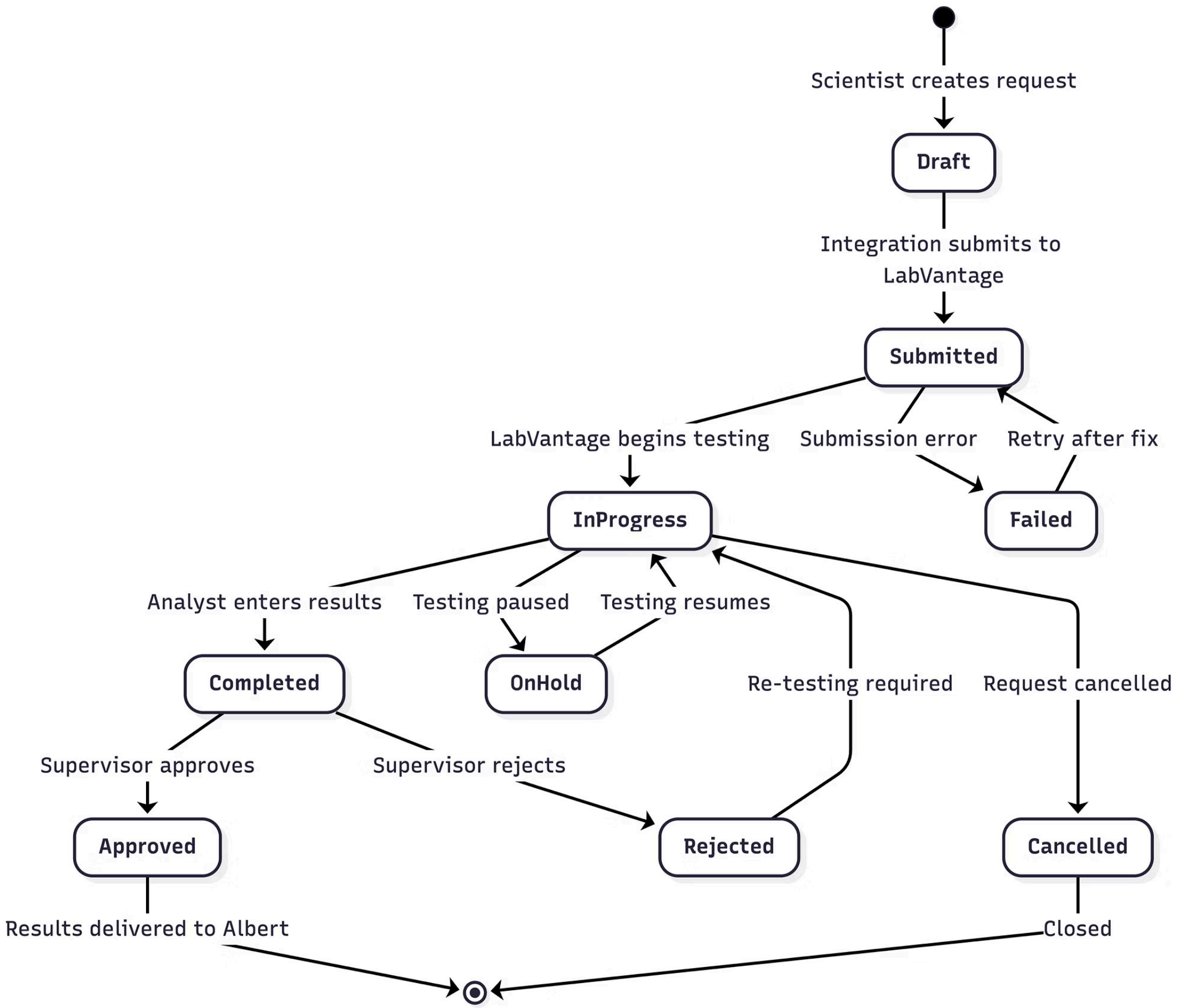
- Lab test turnaround = hours to days. 5-min polling = results appear in Albert within 5 minutes of completion.
- LabVantage has **no native webhooks** — there's no push notification mechanism.
- Polling is simpler to build, test, debug, and operate than event streaming.
- 5-min interval = ~288 API calls/day — well within any reasonable rate limit.

Watermark-Based Deduplication

The poller tracks the timestamp of the last processed result. Each poll asks: "give me everything completed since X." This ensures no results are missed and prevents duplicate processing during retries, ensuring data consistency.

Test Request Lifecycle

State Mapping



What Happens When Things Break

Anticipating and mitigating common failure modes for robust integration.

1	LabVantage Down	HTTP 503 / Timeout	Circuit breaker, queue requests. Auto-retry. Scientist sees "Pending."
2	Duplicate Submission	Idempotency check (query LV by ExternalRef)	Skip creation, link to existing sample.
3	Network Partition (unsaved correlation)	Hourly reconciliation job	Link orphaned samples. Alert on mismatch.
4	Result to Wrong Task	Correlation check (Task ID ↔️ Sample ID)	Route to DLQ, alert for manual review.
5	Albert Down (results stuck in LV)	Poller health check fails	Watermark-based catch-up on recovery. No data loss.

Observability

- Health check endpoint for the integration service.
- Dashboard: request volume, success rate, DLQ depth, poll latency, error rate.
- Alerts: DLQ depth > 0, circuit breaker tripped, poll failure streak > 3.

Integration Approach Decision

Recommendation: Custom Python integration service deployed as a container in Albert's infrastructure.

Why Not iPaaS?

- ❌ No pre-built LabVantage connector exists.
- ❌ High licensing cost: $36K-$180K+/year without proportional value.
- ❌ Doesn't simplify hard problems (polling, state management, idempotency).
- ❌ **Albert doesn't own the IP** – locked to vendor.
- ❌ No contribution to AlbertHub productization.

Why Custom Works

- ✅ Albert owns the code, IP, and operational knowledge.
- ✅ Uses a well-understood tech stack: Python + Albert SDK + LabVantage REST API.
- ✅ Containerized deployment: portable, scalable, testable.
- ✅ Becomes the reference implementation for future connector frameworks.

Cost Comparison: iPaaS incurs $36K-$180K+/year in licensing, while custom development ensures Albert owns the IP with a one-time build investment.

What Discovery Must Answer for LIMS

Addressing critical unknowns upfront to define the integration architecture and scope.

Network Path & Accessibility

Is LabVantage on-prem or cloud-hosted? Can Albert's cloud infrastructure establish secure connectivity (firewall, VPN)?

API Capabilities & Access

Is the LabVantage REST API enabled? What named queries exist? Can we create samples/tests via REST or must we use Actions API?

RESTPolicy Configuration

LabVantage uses deny-by-default security. Who configures it, how long does it take, and what's the process for updates?

Data Mapping Requirements

What fields are mandatory for sample/test creation? What custom fields does the customer utilize, and how should they map?

In-Flight Request Handling

When the integration goes live, how do we manage test requests already in LabVantage? Ignore, backfill, or other strategy?

LabVantage Version

REST API capabilities and features vary significantly by version. Which specific version of LabVantage is the customer running?

Each of these unknowns directly impacts the architecture, and that's why we need discovery.

Increasing Velocity Over Time

10 min

Build the Integration Factory, Not Just Integrations

The Problem Today:

- Every new customer integration is built from scratch.
- Engineering time scales linearly with customer count.
- Knowledge stays in individual engineers' heads.

The Mindset Shift:

Every customer engagement leaves behind TWO things:

1. A working integration for THAT customer
2. A reusable artifact that makes the NEXT customer faster

The AlbertHub Vision:

This is the core of the AlbertHub vision — turning bespoke engineering into repeatable, productized capabilities.

The Three-Phase Productization Strategy

1

Phase 1: Build (This Customer)

- Custom Python integration for LabVantage ↔️ Albert with clean architecture.
- Estimated effort: **~16-22 person-weeks** for the LIMS build.

2

Phase 2: Refactor (After Go-Live)

- Extract generic components into a shared framework (scheduler, retry, monitoring, health checks).
- Isolate LabVantage-specific logic into a `LabVantageAdapter`.
- Externalize field mappings to YAML config files.
- Estimated investment: **~4-6 person-weeks**.

3

Phase 3: Reuse (Next Customer)

- New LabVantage customer: only requires new YAML config + testing.
- New LIMS platform (e.g., STARLIMS): requires new adapter + config + testing.
- Build phase drops from ~16 weeks to **~6 weeks (~60% reduction)**.
- Total engagement drops from ~22 weeks to **~10 weeks (~55% reduction)**.

Key Insight: We don't know the right abstractions until we've built the first one. Premature abstraction is often more costly than refactoring after gaining real-world insights from the initial implementation.

Configuration-Driven, Not Code-Driven

Future integrations will be defined by declarative configuration, minimizing custom code.

```yaml
connector:
  name: customer-acme-labvantage
  type: labvantage
  version: "8.7"

connection:
  host: lv.acme.com
  port: 8443
  auth:
    type: external_app_user
    credentials_secret: acme-lv-credentials

polling:
  interval_seconds: 300
  result_query: "CompletedResultsSinceTimestamp"
  watermark_field: "completiondate"

field_mapping:
  outbound:
    albert_task_id: ExternalReference
    sample_type: s_sampletype
    test_template: testtemplateid
    priority: priority
  inbound:
    result_value: measured_value
    result_unit: unit
    test_status: task_status
    analyst: completed_by

error_handling:
  max_retries: 3
  backoff_multiplier: 2
  circuit_breaker_threshold: 5
```

New customer = write a config file like this + run the test harness. No code changes to the core framework.

Six Artifacts, Ranked by Actual Reuse Value

1	**Connector Framework + Templates**	Shared code: scheduler, retry, DLQ, monitoring. Boilerplate for new adapters.	**High** — the core of productization
2	**Field Mapping Configs**	YAML files per customer (see previous slide)	**High** — new customer = new config file
3	**Contract Test Harness**	Automated tests that validate ANY `LIMSConnector` implementation against the interface contract	**High** — customer-independent, catches regressions
4	**Deployment Playbooks**	Step-by-step: provision environment, configure secrets, deploy container, validate health	**Medium** — useful early, evolves into automation
5	**Monitoring Runbooks**	Standard dashboards and alert rules: DLQ depth, error rate, poll latency	**Medium** — generic monitoring is reusable; thresholds are per-customer
6	**Developer Documentation**	How to build a new adapter, framework internals, API reference	**Medium** — high value IF maintained

```
// The `LIMSConnector` interface (what every adapter must implement):
authenticate() // connect to the external LIMS
submit_request(task) // create a sample/test in the external LIMS
poll_results(since) // retrieve completed results since a watermark
map_inbound(raw_result) → Albert result format
map_outbound(albert_task) → LIMS request format
```

Long-Running Workflows Deserve a Purpose-Built Engine

The integration workflows we're building (polling loops, retry chains, multi-day test lifecycles) are textbook long-running workflows. As AlbertHub scales, maintaining custom orchestration code across dozens of integrations becomes operational overhead.

Temporal is an open-source workflow engine built specifically for this problem:

Temporal

Custom Orchestration

- Custom polling loop + watermark catch-up
- Retry logic with exponential backoff
- State machine tracking request lifecycle
- Reconciliation job for crash recovery
- DLQ + manual review process

What Temporal Provides Natively

- Durable timers that survive restarts
- Configurable activity retry policies
- Workflow state persisted automatically
- Automatic checkpoint and resume
- Built-in failure queue with full execution history

Key Insight: We're not replacing what we build for Customer 1. We're recognizing that the patterns we'll extract in Phase 2 (scheduler, retry engine, state manager) already exist as battle-tested primitives in Temporal. Adopting it means our framework delegates orchestration to a purpose-built engine, and our engineers focus on connector logic and domain mapping.

When to make this move: After Phase 2 (Refactor). Once we know which orchestration patterns are truly universal across integrations, Temporal becomes the natural home for them.

From Engineering Tool to Platform Capability

Before AlbertHub

ENGINEERING

Builds every integration from scratch

TECH OPS

Waits for engineering

CUSTOMER SUCCESS

Escalates every integration request

PARTNERS / SIS

Can't participate

CUSTOMERS

Waits weeks for connectivity

With AlbertHub

ENGINEERING

Builds adapters and framework; reviews configs

TECH OPS

Deploys connectors using playbooks; monitors dashboards

CUSTOMER SUCCESS

Configures field mappings in YAML; runs test harness

PARTNERS / SIS

Builds adapters using documentation and SDK

CUSTOMERS

(Future: self-service via AlbertHub UI)

How I'd share across the team

- Internal demo after each major milestone (biweekly)
- Documentation published to Albert's developer portal
- "Integration Office Hours" — weekly slot for CS/Tech Ops questions
- Post-mortem after first customer go-live → lessons learned feed back into playbooks